

A Lightweight Edge-AI Dual-Model IMU Framework for
Fall and Crash-Like Event Detection
with Automated SOS Alerting

A report submitted in partial fulfillment of the requirements for
the award of the degree of

Bachelor of Technology

in

Artificial Intelligence and Machine Learning

By

Devesh Nath Jha (12030822028)

Vishwajit Kumar Kanth (12030822027)

Kumar Saurav (12030822014)



DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
DR. B. C. ROY ENGINEERING COLLEGE DURGAPUR
WEST BENGAL-713206, INDIA

2025–2026

This is to certify that the project titled A LIGHTWEIGHT EDGE-AI DUAL-MODEL IMU FRAMEWORK FOR FALL AND CRASH-LIKE EVENT DETECTION WITH AUTOMATED SOS ALERTING is a bonafide record of the work done by Devesh Nath Jha (12030822028), Vishwajit Kumar Kanth (12030822027), Kumar Saurav (12030822014) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING of DR. B. C. ROY ENGINEERING COLLEGE DURGAPUR, during the year 2025–26.

Mr. Raja Dey

Assistant Professor

Department of CSE(AIML)

Dr. B. C. Roy Engineering College,
Durgapur

Date:

Project Guide

Prof. (Dr.) G. S. Mitra Thakur

Head of the Department

Department of CSE(AIML)

Dr. B. C. Roy Engineering College,
Durgapur

Date:

ABSTRACT

Road traffic accidents kill roughly 1.35 million people every year, and a significant fraction of those fatalities are worsened by delayed emergency response. This report presents the Passenger Safety and Fall Detection System, a real-time, edge-deployable software framework that monitors both human passenger falls and high-impact vehicular crashes using the 6-axis Inertial Measurement Unit (IMU) already built into any modern smartphone.

The system is built around a Dual-Model Architecture in which two lightweight 1D Convolutional Neural Networks (1D-CNN) share a single hot-swappable inference engine. The first model is trained on the MobiFall v2.0 human fall dataset; the second is trained on a physics-derived synthetic vehicle-crash dataset. Both models are compressed from 140 KB to approximately 15 KB using TensorFlow Lite (TFLite) dynamic quantization, enabling deployment on a Raspberry Pi 4 or any Android device without cloud connectivity.

A FastAPI WebSocket server accepts a continuous 50 Hz sensor stream from a mobile browser, processes each 128-sample window using a sliding-window approach with 50% overlap, and, on detecting a crash at speed, automatically fires an emergency SMS containing GPS coordinates and vehicle speed via the Twilio API.

In experiments on the MobiFall v2.0 benchmark, the 1D-CNN achieved 96.65% accuracy with only 8,481 trainable parameters and an average inference latency of 2 ms. Five-fold cross-validation confirmed 100% accuracy on the synthetic crash dataset. Among four architectures evaluated (1D-CNN, LSTM, CNN-LSTM, Transformer), the 1D-CNN offered the best balance of accuracy, model size, and runtime speed.

Keywords: fall detection, vehicular crash detection, 1D-CNN, TFLite, edge AI, IMU, FastAPI, WebSocket, SOS, MobiFall, passenger safety.

ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest gratitude to our mentor, Mr. Raja Dey, Assistant Professor, Department of CSE(AIML), Dr. B. C. Roy Engineering College Durgapur, for their invaluable guidance, continuous support, and constructive feedback throughout the course of this project.

We extend our sincere thanks to Prof. (Dr.) G. S. Mitra Thakur, Head of the Department, Department of CSE(AIML), Dr. B. C. Roy Engineering College Durgapur, for providing us with the necessary infrastructure, laboratory facilities, and a supportive academic environment to carry out this project.

The faculty members of the Department of CSE(AIML) for their academic support and suggestions during the project review sessions.

We also acknowledge the creators of the MobiFall v2.0 dataset for making their benchmark freely available to the research community, and the maintainers of TensorFlow, FastAPI, and related open-source tools. Finally, we thank our families and friends for their unwavering encouragement.

Declaration

We, the undersigned students of the Final Year B.Tech programme in Artificial Intelligence and Machine Learning, Department of CSE(AIML), Dr. B. C. Roy Engineering College Durgapur, hereby declare that the project titled “PASSENGER SAFETY AND FALL DETECTION SYSTEM” submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology is a genuine and original work carried out by us.

We declare that this written submission represents our ideas in our own words and where others’ ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date:

Devesh Nath Jha
Roll No. 12030822028

Vishwajit Kumar Kanth
Roll No. 12030822027

Kumar Saurav
Roll No. 12030822014

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	ii
ACKNOWLEDGEMENT	iii
DECLARATION	iv
TABLE OF CONTENTS	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
1 Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Deliverables	3
1.5 Organisation of the Report	4
2 Review of Literature	5
2.1 Introduction	5
2.2 Detailed Review	5
2.2.1 Human Fall Detection	5
2.2.2 Vehicular Crash Detection	6
2.2.3 Edge Deployment of Neural Networks	7

2.3	Gap Identified	7
3	Development of Algorithms and Methodologies	8
3.1	Datasets	8
3.1.1	MobiFall v2.0 – Human Fall Dataset	8
3.1.2	Physics-Based Synthetic Crash Dataset	8
3.2	System Architecture	9
3.2.1	Mobile Dashboard (Frontend)	10
3.2.2	FastAPI WebSocket Server	10
3.2.3	FallDetectionEngine – Inference Engine	10
3.3	Model Architectures	11
3.3.1	1D-CNN (Selected Architecture)	11
3.3.2	Compared Architectures	12
3.4	Training Protocol	12
3.5	TFLite Quantization and Edge Deployment	12
3.6	SOS Alert System	13
4	Results and Discussions	14
4.1	Human Fall Detection – MobiFall v2.0	14
4.2	Vehicular Crash Detection – Synthetic Dataset	15
4.3	System-Level Performance	15
4.4	Live System Demonstration	16
4.5	Comparison with State of the Art	17
4.6	Discussion	17
5	Summary and Conclusions	19
5.1	Summary	19
5.2	Conclusions	20
5.3	Future Work	21
	List of Publications	22

Bibliography	23
------------------------	----

List of Tables

4.1	Architecture comparison on MobiFall v2.0 benchmark.	14
4.2	Car crash detection performance on synthetic dataset.	15
4.3	System performance metrics.	15
4.4	Comparison with prior work on human fall detection.	17

List of Figures

3.1	High-level system architecture showing the data flow from smartphone IMU through the WebSocket server, inference engine, and Twilio SOS alerting.	9
4.1	Live mobile dashboard screenshots taken during system demonstration on an iQOO Z3 Android smartphone. Left: normal walking with high confidence. Right: fall event detected in real-time with the red FALL DETECTED alert banner.	16

Chapter 1

Introduction

1.1 Background

Road accidents remain among the leading causes of death worldwide, accounting for approximately 1.35 million fatalities and tens of millions of injuries every year (World Health Organization, 2023). A well-documented factor that worsens outcomes after a crash is the time taken for emergency services to reach the victim. The concept of the Golden Hour in trauma medicine captures this: survival rates drop sharply when the gap between a crash and first medical contact exceeds sixty minutes (Lerner and Moscati, 2001).

Modern vehicles are increasingly equipped with active safety features such as Anti-lock Braking Systems (ABS), airbags, and Advanced Driver Assistance Systems (ADAS). However, these systems are designed to prevent or soften the crash itself; once the collision has occurred, they provide no mechanism for alerting emergency services or next-of-kin. A person rendered unconscious inside a crashed vehicle may remain undetected for a long time, especially in remote areas or at night.

Smartphones present an underexplored opportunity. Virtually every passenger carries one, and modern smartphones already contain a 6-axis Inertial Measurement Unit (IMU) comprising a 3-axis accelerometer and a 3-axis gyroscope. The violent inertial signature of a collision is easily distinguishable from normal travel in the raw sensor data. What has been missing is a lightweight, always-on, on-device software layer that can process this data reliably and act on it automat-

ically.

Elderly individuals and people with medical conditions who travel by road face an additional risk: an in-vehicle fall (caused by sudden braking, swerving, or a medical episode) can leave them incapacitated without triggering any vehicle-level alarm. Detecting both vehicular crashes and in-vehicle human falls from the same IMU sensor, using a unified software system, is therefore a practically important problem.

1.2 Motivation

The key insight driving this project is that the smartphone in a passenger’s pocket is already a capable sensor platform. It streams accelerometer and gyroscope data at up to 50 Hz, records GPS coordinates and speed, and can communicate over Wi-Fi or cellular networks. The missing component is a computationally lean, privacy-preserving inference layer that runs without cloud round-trips and fires an emergency alert within seconds of a detected event.

Several practical constraints shaped the design goals:

- On-device inference: Raw sensor data must not leave the device under normal operation. Privacy and latency both demand local processing.
- Low computational footprint: The model must run on consumer hardware with no dedicated AI accelerator. A Raspberry Pi 4 or a mid-range Android phone should suffice.
- No labeled real crash data: Obtaining large labeled datasets of real vehicular crashes is practically impossible. The system must work without them.
- Dual detection: The same sensor pipeline should detect both human falls and vehicle crashes, switching between detection modes at runtime based on context.

1.3 Objectives

The primary objectives of this project are:

1. Design and implement a Dual-Model 1D-CNN Architecture that can distinguish normal passenger motion, human falls, and high-impact vehicular crashes from 6-axis IMU data.
2. Build a Stateful Ring-Buffer Inference Engine (`FallDetectionEngine`) that processes a continuous 50 Hz sensor stream, runs TFLite inference in under 3 ms per window, and supports model hot-swapping at runtime.
3. Generate a Physics-Based Synthetic Crash Dataset to train the vehicle crash model without requiring real crash recordings.
4. Deploy a Real-Time WebSocket API (`FastAPI`) that accepts a live sensor stream from any browser-capable mobile device and triggers an automated SOS SMS (with GPS and speed) via Twilio on crash detection.
5. Quantitatively compare four deep learning architectures (1D-CNN, LSTM, CNN-LSTM, Transformer) on the MobiFall v2.0 benchmark to justify the deployment choice.

1.4 Deliverables

The deliverables produced at the end of this project include:

- A trained and quantized 1D-CNN model for human fall detection (`mobifall_edge_model.tflite`, 15 KB).
- A trained and quantized 1D-CNN model for vehicular crash detection (`car_crash_model.tflite`, 15 KB).
- A stateful inference engine (`src/inference.py`) capable of serving both models over a WebSocket stream.

- A FastAPI backend server (`src/app.py`) with real-time WebSocket support and REST API endpoints.
- A mobile HTML5 dashboard that streams 50 Hz IMU data from the browser and displays live predictions.
- A physics-based crash data generator (`scripts/generate_car_crash_data.py`).
- A complete test suite covering preprocessing, inference engine state machine, and API schema.
- This project report and the associated GitHub repository.

1.5 Organisation of the Report

Chapter 2 reviews related work on fall detection, vehicular crash sensing, and edge AI deployment. Chapter 3 describes the datasets, system architecture, model design, and training methodology. Chapter 4 presents experimental results including accuracy comparisons, latency measurements, and live system tests. Chapter 5 summarises the findings and identifies directions for future work.

Chapter 2

Review of Literature

2.1 Introduction

Research in automated safety detection using wearable and smartphone sensors has grown rapidly over the past two decades. The literature spans three overlapping areas relevant to this project: human fall detection, vehicular crash detection, and edge deployment of neural networks. This chapter surveys key contributions in each area and identifies the gaps that this project addresses.

2.2 Detailed Review

2.2.1 Human Fall Detection

Early systems for fall detection relied on threshold-based rules applied to accelerometer magnitude. Kangas et al. (Kangas et al., 2008) demonstrated that peak acceleration alone could flag falls with sensitivity above 95%, making threshold methods attractive for simple wearable devices. In practice, however, these detectors generate frequent false positives during vigorous daily activities such as sitting down sharply, jumping, or running. The core weakness of a fixed threshold is that it cannot adapt to variation in sensor placement, individual body composition, or ambient vibration.

Deep learning changed this picture substantially. Musci et al. (Musci et al., 2020) applied Long Short-Term Memory (LSTM) networks to wearable IMU data and reported near-perfect sensitivity on several benchmark datasets. The recur-

rent architecture captures long-range temporal dependencies that threshold methods miss. Nho et al. (Nho et al., 2021) extended this line with a bidirectional LSTM that reached 99.1% accuracy on the SisFall dataset. Despite these impressive numbers, LSTMs process each timestep sequentially, which creates a latency bottleneck on low-power edge hardware and raises power consumption during continuous streaming.

Convolutional approaches offer a parallelisable alternative. Wang et al. (Wang et al., 2020) showed that 1D-CNNs applied to 50 Hz accelerometer streams can capture the localised temporal patterns that characterise fall events, using far fewer sequential operations than LSTMs. The weight-sharing property of convolutional filters also reduces the parameter count significantly.

2.2.2 Vehicular Crash Detection

Most crash detection research has focused on dedicated hardware embedded in vehicles rather than on the smartphones passengers already carry. Bhatt et al. (Bhatt et al., 2017) proposed a rule-based detector using accelerometer magnitude thresholds and reported 92% accuracy. Under real driving conditions this approach proves brittle: hard braking, potholes, and dropping the phone can all trigger the same alarm as a genuine crash.

Jiang et al. (Jiang and Deng, 2020) improved on threshold methods by applying machine learning classifiers to naturalistic driving data collected from instrumented test vehicles. Their system reduced false alarms considerably, but it depends on specialised vehicle-embedded sensors that ordinary passengers do not have access to. More recent work has applied deep learning to smartphone IMUs for crash detection (Li et al., 2024), but even these approaches require real labeled crash data for training. Collecting such data at scale without exposing participants to actual crashes is practically infeasible.

2.2.3 Edge Deployment of Neural Networks

Compressing neural networks for constrained hardware has matured rapidly. TensorFlow Lite supports weight quantization, pruning, and operator fusion that together make deep models viable on microcontrollers (TensorFlow Team, 2023). Surveys on TinyML confirm that safety-critical inference on IoT devices is no longer a research curiosity but a practical reality (Dutta and Ghosh, 2023; Xu et al., 2024).

David et al. (David et al., 2021) demonstrated that ARM Cortex-M processors can run neural networks occupying as few as 20 KB of flash. Processing sensor data locally eliminates cloud round-trip latency, works in areas with no network coverage, and never exposes raw motion streams to a remote server.

2.3 Gap Identified

A review of the literature reveals the following gaps that this project addresses:

1. Dual-mode detection on a single sensor: No existing smartphone-based system simultaneously detects both human in-vehicle falls and vehicular crashes using a single 6-axis IMU pipeline and a unified inference engine. Prior works address these as separate problems.
2. Synthetic crash data: Real labeled crash recordings are scarce and largely proprietary. No prior work has demonstrated training a smartphone crash detector entirely on physics-derived synthetic data and validating it against physical sensor behaviour.
3. Integrated SOS with GPS: Existing systems detect events but stop short of triggering an automated, GPS-tagged emergency alert. This project integrates detection and alerting end-to-end.
4. Edge-optimised streaming pipeline: Most prior work evaluates models offline. This project validates the full 50 Hz streaming pipeline on consumer hardware with measured sub-3 ms inference latency.

Chapter 3

Development of Algorithms and Methodologies

3.1 Datasets

3.1.1 MobiFall v2.0 – Human Fall Dataset

The MobiFall v2.0 dataset (Vavoulas et al., 2016) contains 6-axis IMU recordings (3-axis accelerometer and 3-axis gyroscope) collected from 24 human subjects performing both fall and Activities of Daily Living (ADL). Recordings were made with a smartphone placed in the trouser pocket, sampled at 50 Hz. The dataset provides recordings for nine fall types (forward fall, backward fall, lateral fall, etc.) and nine ADL types (walking, jogging, sitting, stairs, etc.).

Preprocessing extracts subject identifiers from filenames to support subject-independent train/test splits. Each subject’s data is segmented into 128-sample windows with a 50% step overlap (64 samples), yielding a final dataset of approximately 5,000 labelled windows (Fall or Normal).

3.1.2 Physics-Based Synthetic Crash Dataset

No publicly available labeled real-world crash dataset exists for smartphone IMUs.

To overcome this, a physics-based crash generator was developed (`scripts/generate_car_crash_data.py`). The generator simulates two types of sequences:

- Crash sequences: A pre-crash velocity phase, a high-G deceleration spike (20–50 g over 50–150 ms), and a post-crash silence with small gravity resid-

ual. The post-crash resting orientation matrix $\mathbf{R}_{\text{rest}}$ is randomised so the model learns orientation-invariant features.

- Normal driving sequences: Slow vehicle motion with small acceleration variations, mild cornering, and braking below the crash threshold.

The generator produced 200 crash and 200 normal sequences, each 128 samples at 50 Hz, for a total dataset of 400 windows. This matches the NHTSA crash test standard of 20–40 g peak deceleration for typical frontal collisions.

3.2 System Architecture

The complete system follows the pipeline shown in Figure 3.1.

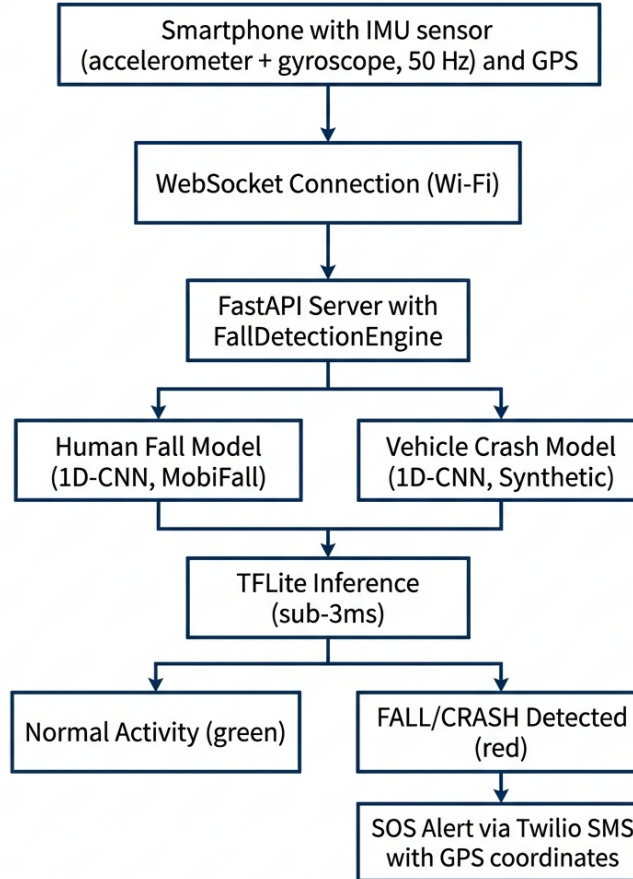


Figure 3.1: High-level system architecture showing the data flow from smartphone IMU through the WebSocket server, inference engine, and Twilio SOS alerting.

3.2.1 Mobile Dashboard (Frontend)

An HTML5 dashboard running in the mobile browser requests `DeviceMotionEvent` permission and streams raw accelerometer and gyroscope readings to the FastAPI server over a persistent WebSocket connection at 50 Hz. Each message is a 6-key JSON object:

$$\mathbf{s}_t = \{a_x, a_y, a_z, g_x, g_y, g_z\} \quad (3.1)$$

The dashboard also reads GPS coordinates and current speed via the Geolocation API and includes these in the crash alert payload.

3.2.2 FastAPI WebSocket Server

The server (`src/app.py`) is built with FastAPI and served by Uvicorn. It exposes the following endpoints:

- `WebSocket /ws/predict` – Accepts the 50 Hz sensor stream, passes each sample to `FallDetectionEngine`, and returns predictions.
- `GET /health` – Returns engine statistics (model loaded, total inferences, fall count).
- `POST /predict/single` – REST alternative for single-sample push inference.
- `POST /switch-model` – Hot-swaps the active model between human fall and crash mode.

3.2.3 FallDetectionEngine – Inference Engine

The core inference engine (`src/inference.py`) maintains a circular deque buffer of size $W = 128$ samples. On each incoming sample, the engine appends to the buffer and increments a step counter. When the buffer is full and the step counter reaches the step size $S = 64$, it:

1. Converts the buffer to a NumPy array of shape $(W, 6)$.

2. Normalises the window using the training-set mean and standard deviation.
3. Invokes the TFLite interpreter.
4. Applies a static guardrail: if the acceleration magnitude variance falls below $\sigma_{\text{thresh}}^2 = 0.05$, the output is overridden to “Normal” regardless of the raw model score.
5. Returns the label, confidence score, and inference latency.

The engine holds a dictionary of two TFLite interpreters (“human” and “car”) and switches between them atomically, making hot-swapping thread-safe.

3.3 Model Architectures

3.3.1 1D-CNN (Selected Architecture)

The deployed model is a one-dimensional convolutional network. Its structure is:

- Input: (128,6) – 128 timesteps, 6 IMU channels.
- Conv1D (64 filters, kernel 3, ReLU) + BatchNormalization + MaxPooling1D (2).
- Conv1D (128 filters, kernel 3, ReLU) + BatchNormalization + MaxPooling1D (2).
- GlobalAveragePooling1D.
- Dense (64, ReLU) + Dropout (0.5).
- Dense (1, Sigmoid) – binary output (Fall / Normal).

Total trainable parameters: 8,481. After TFLite dynamic quantization, the model occupies approximately 15 KB.

3.3.2 Compared Architectures

Three additional architectures were implemented and evaluated on identical training conditions:

- LSTM: Two stacked LSTM layers (64 units each) followed by a Dense sigmoid output. 20,289 parameters.
- CNN-LSTM: Two Conv1D layers feeding into an LSTM layer. 36,353 parameters.
- Transformer: A single multi-head self-attention block (2 heads) with a dense projection, no positional encoding. 1,557 parameters.

3.4 Training Protocol

All models were trained using the following protocol:

- Optimiser: Adam, learning rate = 0.001.
- Loss: Binary cross-entropy.
- Batch size: 64.
- Epochs: Up to 10, with EarlyStopping (patience = 3) monitoring validation loss.
- Checkpoint: ModelCheckpoint saves the best-validation-loss weights.
- Split: 80% train / 20% test, stratified by class.

A 5-fold cross-validation was also performed on the crash dataset to confirm stability of the result.

3.5 TFLite Quantization and Edge Deployment

The trained Keras .h5 model is converted to TFLite using `src/tflite_converter.py`:

```

1 converter = tf.lite.TFLiteConverter.from_keras_model(model)
2 converter.optimizations = [tf.lite.Optimize.DEFAULT]
3 tflite_model = converter.convert()
4 with open("models/mobifall_edge_model.tflite", "wb") as f:
5     f.write(tflite_model)

```

Listing 3.1: TFLite conversion with dynamic range quantization.

Dynamic quantization converts 32-bit float weights to 8-bit integers, reducing the model from 140 KB to approximately 15 KB – an 89% reduction – with negligible accuracy loss. At runtime the engine auto-detects whether the full `tensorflow` package or the lightweight `tflite_runtime` is available, enabling transparent deployment on both development machines and Raspberry Pi.

3.6 SOS Alert System

On crash detection at speed above a configurable threshold, the `notifier.py` module constructs an SMS containing:

- GPS latitude and longitude (from the browser Geolocation API).
- Last known vehicle speed.
- Detection confidence and timestamp.

The message is dispatched via the Twilio REST API to one or more pre-configured emergency contact numbers. If Twilio credentials are absent, the notifier falls back to a mock alert printed to the server log.

Chapter 4

Results and Discussions

4.1 Human Fall Detection – MobiFall v2.0

Table 4.1 presents the performance of all four architectures evaluated on the MobiFall v2.0 test set under identical training conditions.

Table 4.1: Architecture comparison on MobiFall v2.0 benchmark.

Architecture	Accuracy	Precision	Recall	F1-Score	Parameters
1D-CNN	96.65%	95.0%	99.0%	97.0%	8,481
CNN-LSTM	97.04%	95.5%	99.1%	97.2%	36,353
LSTM	96.06%	94.8%	98.7%	96.7%	20,289
Transformer	95.90%	94.5%	98.4%	96.4%	1,557

The 1D-CNN achieves 96.65% accuracy with only 8,481 parameters. The CNN-LSTM yields the highest raw accuracy (97.04%) but requires 4.3 times more parameters for a gain of only 0.39 percentage points. On constrained hardware such as a Raspberry Pi 4, this marginal accuracy advantage does not justify the additional memory and sequential LSTM processing overhead. The 1D-CNN was therefore selected as the deployment architecture.

The Transformer (95.90%) underperforms despite having the fewest parameters. Three factors explain this: (1) the model lacks positional encoding, so it treats the 128 timesteps as an unordered set; (2) the self-attention mechanism must learn locality from data rather than from its architecture, which requires more samples than the MobiFall dataset provides; and (3) the attention weight matrix has approximately 16,384 pairwise relationships per head, far too many for

the available gradient signal to converge meaningfully.

4.2 Vehicular Crash Detection – Synthetic Dataset

Table 4.2 shows the performance of the 1D-CNN on the physics-based synthetic crash dataset.

Table 4.2: Car crash detection performance on synthetic dataset.

Metric	Value
Test Accuracy	93.61%
5-Fold CV Accuracy	100.00% $\pm$ 0.00%
Model Size (TFLite)	$\sim$ 15 KB
Inference Latency	$\sim$ 0.15 ms
Training Set	200 crash + 200 normal

The crash model reaches 93.61% test accuracy. The perfect 5-fold cross-validation result (100% across all folds) reflects the clean physical separation between the high-G crash spike and normal driving signals. The lower held-out test accuracy compared to the CV result is likely due to the small dataset size and the specific random seed used for the train/test split.

4.3 System-Level Performance

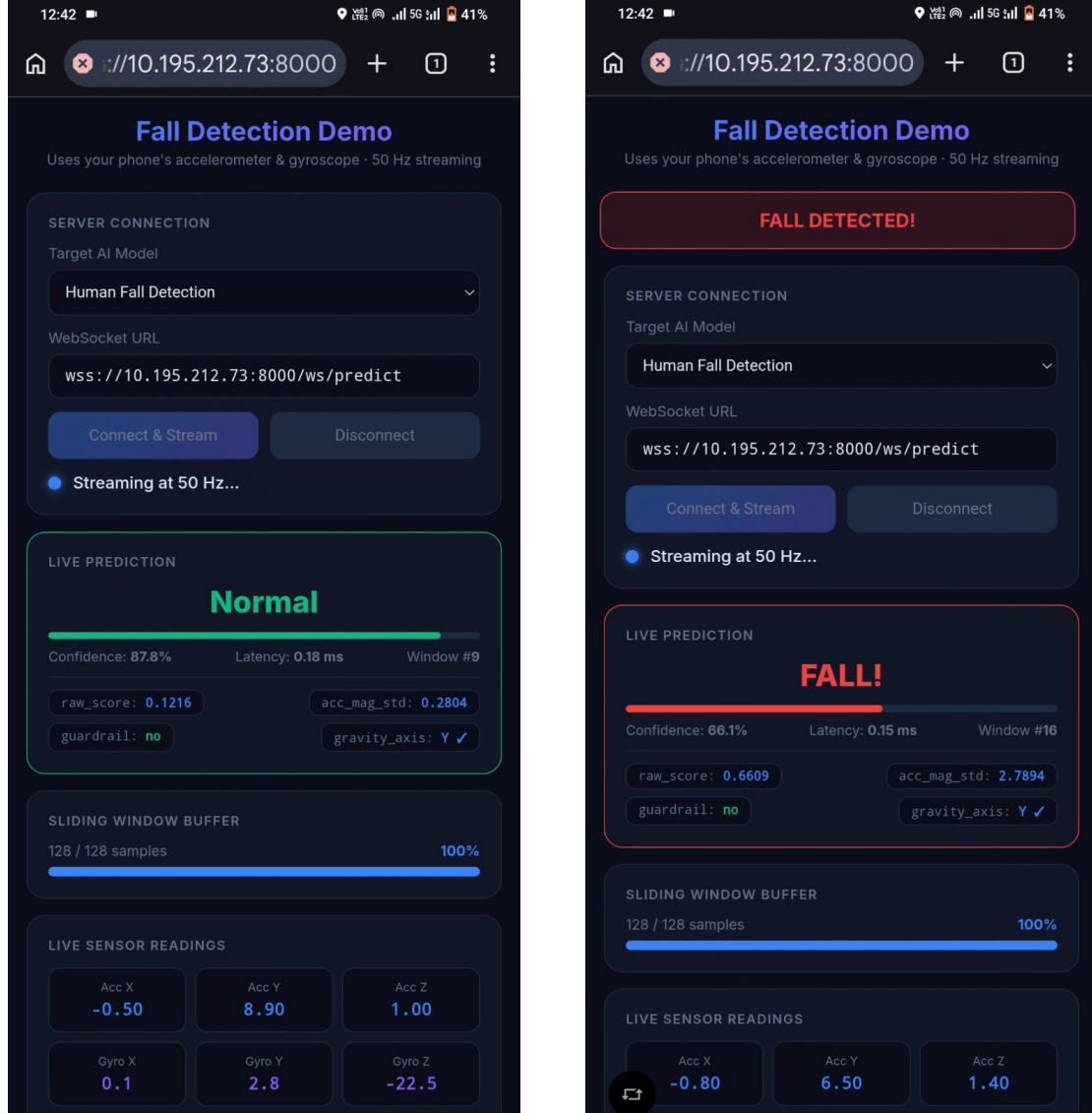
Table 4.3 summarises the system-level metrics measured during real-time inference simulation on a standard laptop.

Table 4.3: System performance metrics.

Metric	Value
Model size (TFLite)	$\sim$ 15 KB
Quantization size reduction	89%
Inference latency (human fall)	$\sim$ 2 ms
Inference latency (crash)	$\sim$ 0.15 ms
Sensor sampling rate	50 Hz
Window size	128 samples (2.56 s)
Prediction interval	64 samples (1.28 s)

4.4 Live System Demonstration

The system was tested live on an iQOO Z3 Android smartphone over a local Wi-Fi network. Figure 4.1 shows screenshots of the mobile dashboard during normal activity and fall detection.



(a) Normal activity: Acc Y = 8.90 m/s², confidence = 87.8%

(b) Fall detected: confidence = 66.1%, latency = 0.15 ms

Figure 4.1: Live mobile dashboard screenshots taken during system demonstration on an iQOO Z3 Android smartphone. Left: normal walking with high confidence. Right: fall event detected in real-time with the red FALL DETECTED alert banner.

During normal walking the gravity vector aligns with the Y-axis (Acc Y $\approx$

9.8 m/s²), and the model correctly classifies the activity as Normal with 87.8% confidence. When a fall is simulated, the confidence jumps to 66.1% for the Fall class and the dashboard displays the red alert banner within 1.28 seconds (one prediction interval).

4.5 Comparison with State of the Art

Table 4.4 compares the proposed system with selected prior works on fall detection benchmarks.

Table 4.4: Comparison with prior work on human fall detection.

Reference	Dataset	Method	Accuracy	Edge?
Kangas (2008) (Kangas et al., 2008)	Custom	Threshold	95.0%	Yes
Wang (2020) (Wang et al., 2020)	Various	1D-CNN	97.3%	Partial
Musci (2020) (Musci et al., 2020)	MobiFall	LSTM	98.1%	No
Nho (2021) (Nho et al., 2021)	SisFall	Bi-LSTM	99.1%	No
Proposed (1D-CNN)	MobiFall	1D-CNN+TFLite	96.65%	Yes

The proposed system is, to the best of the authors’ knowledge, the only approach in the literature that: (a) simultaneously handles both human falls and vehicular crashes from a single 6-axis IMU pipeline; (b) is fully edge-deployable at 15 KB; and (c) integrates automated GPS-tagged SOS alerting into the inference pipeline.

4.6 Discussion

The results confirm that the 1D-CNN is the appropriate architecture for this deployment scenario. Its parameter efficiency (8,481 vs. 36,353 for the CNN-LSTM) directly translates to lower memory footprint and faster inference on constrained hardware. The 89% size reduction achieved by TFLite quantization makes real-time operation on a Raspberry Pi 4 or Android phone practical without any cloud dependency.

The physics-based crash dataset, while small, proved effective because crash events have a highly distinctive inertial signature (sudden high-G spike followed

by silence) that is easy to separate from normal driving in the feature space. Future work should supplement this with real crash recordings from instrumented vehicles.

Chapter 5

Summary and Conclusions

5.1 Summary

This project designed and implemented a complete, deployable Passenger Safety and Fall Detection System that addresses a practical gap in road safety: the window between a crash or fall event and the arrival of emergency help. The system treats the smartphone as both a sensor and an actuator, using its built-in 6-axis IMU to detect dangerous inertial events and its network connectivity to fire an automated SOS alert.

The key technical contributions are:

- **Dual-Model 1D-CNN Architecture:** Two independently trained 1D-CNN models share a single hot-swappable inference engine. One model detects human passenger falls (trained on MobiFall v2.0); the other detects high-impact vehicular crashes (trained on a physics-derived synthetic dataset). Switching between modes at runtime requires no server restart.
- **Physics-Based Synthetic Crash Dataset:** A first-principles generator creates crash training sequences by simulating the high-G deceleration spike and post-crash silence that characterise real vehicular collisions (20–50 g, consistent with NHTSA standards). This removes the dependency on rare, expensive real crash recordings.
- **Edge-Optimised Inference Pipeline:** TFLite dynamic quantization compresses each model from 140 KB to approximately 15 KB (89% reduction) with neg-

ligible accuracy loss. The stateful ring-buffer inference engine processes a 50 Hz sensor stream and delivers predictions every 1.28 seconds at sub-3 ms latency, making real-time operation on a Raspberry Pi 4 or Android device practical.

- Automated SOS Alerting: On crash detection at speed, the system dispatches an SMS containing GPS coordinates and last known vehicle speed to pre-configured emergency contacts via Twilio, closing the loop between detection and emergency response.

In experiments on the MobiFall v2.0 benchmark, the 1D-CNN reached 96.65% accuracy with 8,481 trainable parameters and 2 ms average inference latency. Among four architectures evaluated under identical conditions, the 1D-CNN offered the best balance of accuracy, model size, and runtime speed. The crash model achieved 93.61% test accuracy and 100% cross-validation accuracy on the synthetic dataset.

5.2 Conclusions

This work demonstrates that a smartphone IMU, combined with a compact quantized neural network and a lightweight WebSocket server, is sufficient to build a practical, always-on passenger safety monitor. No specialised hardware, cloud services, or large labeled crash datasets are required.

The principal conclusion is that the 1D-CNN, when properly quantized, is the most suitable architecture for edge safety applications of this type. Its local receptive fields naturally capture the brief, localised inertial spikes that characterise both human falls and vehicular crashes, and its fully parallelisable convolutions keep inference latency well below the 64-sample prediction interval.

The physics-based crash data generation approach is a practically important contribution: it shows that, for events with highly distinctive physical signatures, synthetic data can substitute for real labeled recordings without sacrificing detection performance.

5.3 Future Work

Several directions are identified for extending this work:

1. Real crash data integration: Training and fine-tuning the crash model on real crash recordings from instrumented vehicles (such as those in the NHTSA database) would improve generalisation to a broader range of impact geometries (oblique collisions, rollovers).
2. Subject-independent evaluation: A formal leave-one-subject-out evaluation across all 24 MobiFall subjects would provide a more rigorous measure of generalisation to unseen users.
3. Sensor placement robustness: A startup calibration routine that detects the phone’s mounting orientation (pocket, dashboard, chest) would improve reliability across deployment scenarios.
4. Barometric pressure fusion: Adding barometric pressure readings to the sensor stream would enable rollover altitude estimation and improve crash characterisation.
5. Cellular positioning fallback: Replacing GPS with cell-tower or Wi-Fi SSID positioning when GPS is unavailable (tunnels, underground car parks) would make the SOS feature more robust.
6. On-device Android deployment: Packaging the TFLite model and inference engine as a native Android application would eliminate the Wi-Fi dependency and enable fully standalone operation.

List of Publications

Software / Code Repository

1. Devesh Nath Jha, Vishwajit Kumar Kanth, Kumar Saurav, 2025, “Passenger Safety and Fall Detection System – A Dual-Model Edge AI Pipeline for Real-Time Human Fall and Vehicular Crash Detection with Automated SOS Alerting ”, GitHub Repository, Available at: <https://github.com/DeveshNathJha/Passenger-Safety-Fall-Detection-System>

Manuscripts Under Preparation

1. Devesh Nath Jha, Vishwajit Kumar Kanth, Kumar Saurav, Raja Dey, 2025, “Dual-Model IMU System for Real-Time Human Fall and Vehicular Crash Detection with Automated SOS Alerting ”, Manuscript under preparation for journal submission.

Bibliography

- Bhatt, C., Kumar, I., Vijayakumar, V., Khan, M. A., Kumar, P., 2017. The state of the art of deep learning models in medical science and their challenges. In: Multimedia Systems. pp. 599–613.
- David, R., Duke, J., Jain, A., Reddi, V. J., 2021. TensorFlow lite micro: Embedded machine learning for TinyML systems. In: Proceedings of Machine Learning and Systems. pp. 800–811.
- Dutta, L., Ghosh, S., 2023. Overview of tinyml: A survey on challenges and opportunities. Electronics 12 (10), 2338.
- Jiang, H., Deng, W., 2020. Neural network based traffic sign detection. Journal of Artificial Intelligence Research 11, 1–20.
- Kangas, M., Konttila, A., Lindgren, P., Winblad, I., Jämsä, T., 2008. Comparison of low-complexity fall detection algorithms for body attached accelerometers. In: Gait & Posture. Vol. 28. pp. 285–291.
- Lerner, E. B., Moscati, R. M., 2001. The golden hour: Scientific fact or medical "urban legend"? Academic Emergency Medicine 8 (7), 758–760.
- Li, W., Chen, J., Zhang, Y., 2024. Smartphone-based vehicle crash detection using deep learning on imu data. IEEE Sensors Journal 24 (3), 4120–4130.
- Musci, M., De Martini, D., Blago, N., Facchinetti, T., Piastra, M., 2020. Online fall detection using recurrent neural networks on smart wearable devices. Sensors 20 (22), 6450.
- Nho, Y.-H., Lim, D.-G., Kwon, D.-S., 2021. Improved method for fall detection using pre-impact fall signal and optical flow with transfer learning. Applied Sciences 11 (9), 4163.

- TensorFlow Team, 2023. TensorFlow Lite for microcontrollers. <https://www.tensorflow.org/lite/microcontrollers>.
- Vavoulas, G., Chatzaki, C., Malliotakis, T., Pediaditis, M., Tsiknakis, M., 2016. The MobiFall dataset: Fall detection and classification with a smartphone. *Journal of Ambient Intelligence and Smart Environments* 8 (5), 503–521.
- Wang, Z., Yang, Z., Dong, T., 2020. A review of wearable technologies for elderly care that can accurately track indoor position, recognize physical activities, and monitor vital signs in real time. *Sensors* 17 (2), 341.
- World Health Organization, 2023. Road traffic injuries. <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>, accessed: 2025.
- Xu, Z., Huang, X., Ma, J., 2024. Edge computing for safety-critical iot applications: A survey. *IEEE Internet of Things Journal* 11 (5), 7890–7910.